**Course Code: CSA1741 Course Name: ARTIFICIAL INTELLIGENCE**
**Slot: B**

## ASSIGNMENT 5 – CO5

**Nature of Assignment:** Individual **Duration:** 1 Week

**Total Marks:** 50 (Part A – 25, Part B – 25)

**Student Name: Niranjna devi S I** **Reg. No:192519100**

**CO5 Coverage:** Agent architecture and its types – Swarm systems – Intelligent systems: representing and using domain knowledge, expert-system shells, explanation, knowledge acquisition – Work with PySpark and RDD – AI chatbot – Real-time emotion detection – AI-powered smart controller – AI-based intrusion detection – Voice assistant – Edge-AI systems – Disaster detection – Deepfake detection and face-authenticity verification – Autonomous drone.

### Purpose of the Assignment

Part A asks you to **design** intelligent systems on paper: architectures, decision rules, pseudocode and traces. Part B asks you to **engineer** them: implement, instrument, stress, compress and defend a deployment decision. The two parts are connected: Part B implements and tests what you designed in Part A.

The central question is:

*How should an AI engineer move from a working model to a deployable agent that works within limits of time, memory, data volume and risk?*

---

## PART A – DESIGN AND REASONING (25 Marks)

No compiler is to be used in Part A. All pseudocode must follow the SIMATS Engineering – Handbook for Algorithmic Pseudocode.

### TASK A1 – Design an Agent Architecture (7 Marks)

**Problem Context – Autonomous Parcel-Delivery Drone**

A drone must deliver medical parcels across a city grid. It must avoid no-fly zones and sudden obstacles, cope with wind drift and return or land safely when the battery is low.

**A1.1 – Model the Agent Problem | 1 Mark**

Specify the PEAS description, classify the environment, and state which agent type(s) (simple reflex, model-based, goal-based, utility-based, learning) the problem requires and why.

**A1.2 – Define Percepts, Internal State and Actions | 1 Mark**

Define exactly what the drone senses, what it must remember between steps and which actions it may take.

**A1.3 – Define the Decision Rule and Priorities | 1 Mark**

State how conflicting goals are resolved (for example, safety over mission over battery economy) and what utility, if any, trades them off.

**A1.4 – Design Complete Pseudocode | 3 Marks**

Write pseudocode of the sense–decide–act cycle for a layered or hybrid architecture. It must show the reactive safety layer, the deliberative planning layer, the arbitration between them, state update and termination. Writing "plan the route appropriately" is not sufficient.

**A1.5 – Trace the Agent | 1 Mark**

Manually trace at least six time steps in which an obstacle suddenly appears and the battery falls below a threshold. Show percept, internal state, layer that acted and action chosen at each step.

**Evidence Expected:** PEAS and agent-type justification, percept/state/action definitions, priority rule, complete pseudocode and a decision trace.

**TASK A2 – Don't Centralise What Can Be Local (5 Marks)**

**Problem Context – Flood Search-and-Rescue with 50 Drones**

A control station currently assigns every drone its search cell. Communication is unreliable and drones fail during missions.

**A2.1 – Diagnose the Weakness of the Centralised Design | 1 Mark**

Identify what happens to coverage, communication load and mission success when the controller or links fail.

**A2.2 – Redesign Using Local Rules | 2 Marks**

Design a decentralised swarm strategy (for example, pheromone-like marking, frontier selection or separation/alignment/cohesion rules). Give the per-agent rule in pseudocode and state what information an agent needs from its neighbours.

**A2.3 – Demonstrate the Emergent Behaviour | 1 Mark**

On a 5 × 5 grid with three drones, manually trace several moves to show how the local rule produces coverage without central assignment.

**A2.4 – Challenge Your Rule | 1 Mark**

Construct a configuration in which your rule causes clustering, deadlock or redundant coverage. Predict the behaviour first, then trace it.

**Evidence Expected:** Diagnosis, per-agent rule, hand trace and the student-designed failure configuration.

**TASK A3 – Discover the Expert-System Architecture Pattern (5 Marks)**

Three systems are described:

1. A pump-fault diagnosis advisor (symptoms → fault)
2. A crop-disease advisor (observations → disease → treatment)
3. A flood-relief triage advisor (reports → priority)

**A3.1 – Identify the Common Pattern | 2 Marks**

Write a small rule set (at least four rules each) for the three domains. Identify what is common to all three and what is domain-specific. Which parts could be reused without changing any code?

**A3.2 – Design a Generic Expert-System Shell | 2 Marks**

Specify the components (knowledge base, working memory, inference engine, explanation facility, knowledge-acquisition interface), the data each exchanges and pseudocode of the inference cycle including conflict resolution.

**A3.3 – Explain the Role of Explanation and Knowledge Acquisition | 1 Mark**

Explain why the explanation facility (HOW and WHY) and the knowledge-acquisition interface are essential for a system used by domain experts who are not programmers.

**Evidence Expected:** Three rule sets, common-versus-specific analysis, shell design with pseudocode and the explanation/acquisition reasoning.

**TASK A4 – Design a Distributed Data Pipeline with RDDs (4 Marks)**

A city's air-quality agent receives about 10 million sensor readings (sensorId, timestamp, value) per day and needs the per-sensor mean and maximum, and the ten noisiest sensors.

Design the pipeline in pseudocode using RDD operations. You must identify:

- Which operations are transformations and which are actions, and when evaluation actually happens
- Which transformations are narrow and which require a shuffle
- Where caching would help and where it would waste memory
- How lineage allows recovery when a partition is lost

Writing "process the data in parallel" is not sufficient.

**Evidence Expected:** Complete pipeline pseudocode, narrow/wide classification, caching decision and lineage explanation.

**TASK A5 – Design When the Full Model Is Too Expensive (4 Marks)**

**Problem Context – Detection Model on an Edge Device**

A detection model (for intrusion, emotion, deepfake or disaster detection; choose one) works well on a server but must run on a device with limited memory and a latency budget of 50 ms per sample.

**A5.1 – Full-Model Perspective | 1 Mark**

Estimate the parameters, memory and operations per sample of your full model and state why it exceeds the budget.

**A5.2 – Design the Compressed Pipeline | 2 Marks**

Design an approximation (for example, pruning, quantisation, a smaller architecture, feature reduction or an early-exit cascade). Give pseudocode of the inference pipeline and state which operations are saved.

**A5.3 – What Have We Sacrificed? | 1 Mark**

Identify precisely what accuracy, robustness or explainability may be lost and how you would detect it.

**Evidence Expected:** Resource estimate, compressed-pipeline pseudocode and an analysis of the trade-off.

---

## PART B – IMPLEMENTATION, INSTRUMENTATION AND DEPLOYMENT DECISIONS (25 Marks)

Python is permitted in Part B. For every task submit source code, test input, expected behaviour, actual output, performance metrics and interpretation. Do not stop when the program runs.

### TASK B1 – Implement and Instrument the Drone Agent (7 Marks)

Implement your Task A1 architecture and Task A2 swarm rule in a grid simulator.

**B1.1 – Validate the Implementation | 2 Marks**

Test at least three scenarios: normal flight, a sudden obstacle, and a boundary case (battery exhausted or goal blocked). State what each scenario verifies and show the behaviour log.

**B1.2 – Compare Design with Implementation | 1 Mark**

Compare your Task A1.5 hand trace with the program's trace and explain every difference.

**B1.3 – Improve the Design | 2 Marks**

Introduce swarm coordination for $N$ drones using your Task A2 rule (or an improved reactive/deliberative arbitration). State what the improvement targets.

**B1.4 – Measure the Effect | 2 Marks**

Over at least 30 seeds, compare single-drone and swarm operation on mission success rate, coverage percentage against time, redundancy ratio, battery used, decision latency and robustness when 0%, 10% and 30% of the drones fail. Test $N = 5$, 10, 25 and 50.

**Evidence Expected:** Simulator and agent code, scenario logs, trace comparison, before/after metric tables and graphs, and an interpretation of the scaling behaviour.

**TASK B2 – Engineer an Adversarial Input for the Expert System (5 Marks)**

Implement your Task A3 shell with a pump-fault rule base (at least 15 rules and 5 fault types) with HOW and WHY explanations.

**B2.1 – Implement and Evaluate | 2 Marks**

Write the rules using only 15 of 30 expert-labelled cases and keep the other 15 as a hold-out set. Report diagnosis accuracy, per-fault precision and recall, rules fired per case and sample explanations.

**B2.2 – Engineer Adversarial Cases | 2 Marks**

Design inputs that expose weaknesses: conflicting rules, missing evidence and dependence on rule order. Predict the outcome of each before running, then compare at least two conflict-resolution strategies (specificity, priority or recency).

**B2.3 – Knowledge Acquisition Test | 1 Mark**

Add or edit a rule at run time without changing engine code and show, with a regression run, which earlier diagnoses were affected.

**Evidence Expected:** Shell code, hold-out results, adversarial case table with predictions and outcomes, strategy comparison and the regression result.

**TASK B3 – Applied Case: Full Model versus Edge Model (8 Marks)**

Choose **one** applied case and one public dataset or simulator (cite the source): **(a)** AI-based intrusion detection, **(b)** real-time emotion detection, **(c)** deepfake detection and face-authenticity verification, **(d)** disaster detection, **(e)** AI-powered smart controller, **(f)** another edge-AI system approved by the instructor. Use the case chosen in Task A5.

**B3.1 – Correctness on a Small Benchmark | 2 Marks**

Build the full model with a leakage-safe split (time-ordered for intrusion data; subject-wise or source-wise for emotion and deepfake data). Report accuracy, precision, recall, F1, ROC-AUC or PR-AUC and the confusion matrix. For case (e) report settling time, overshoot, energy use and constraint violations against a rule-based baseline.

**B3.2 – Find a Counterexample | 2 Marks**

Find and categorise at least five mis-predictions. Identify a class of inputs on which the model fails (for example, an unseen attack type, unseen generator, poor lighting or sensor noise) and demonstrate it.

**B3.3 – Increase the Problem Size | 2 Marks**

Using the Task A4 RDD pipeline in PySpark, process 10^5, 10^6 and 10^7 records (feature extraction or batch scoring). Record runtime against partitions {1, 2, 4, 8}, the effect of caching and the number of shuffles. Explain any case where more partitions did not improve speed.

**B3.4 – Engineering Trade-Off | 2 Marks**

Implement the Task A5 compressed model. Report latency per sample (mean and 95th percentile), throughput, model size and the accuracy change against the full model. State whether the 50 ms budget is met.

**Evidence Expected:** Code, split description, metric tables, error categories, scaling table, compression comparison and interpretation.

**TASK B4 – Final Engineering Decision: Conversational Interface and Deployment (5 Marks)**

Operators want to ask the Task B3 system questions through a chatbot or voice assistant (for example, "Which alerts are open?", "Why was this flagged?").

**B4.1 – Implement a Practical Strategy | 2 Marks**

Build a retrieval-based chatbot with at least 8 intents (TF-IDF with cosine similarity or a classifier) and an out-of-scope fallback controlled by a confidence threshold. Report per-intent precision, recall and F1, confusion matrix and response latency.

**B4.2 – Challenge Your Strategy | 1 Mark**

Simulate speech-recognition errors by corrupting test utterances at word-error rates of 0%, 10%, 20% and 30%. Report intent accuracy, out-of-scope detection and false-accept rate, and show a threshold-sweep curve.

**B4.3 – Production Decision | 2 Marks**

Decide whether the detection model and the assistant should run on the edge device or in the cloud. Defend the decision using your own latency, accuracy, throughput, privacy and cost evidence, and state the conditions under which you would reverse it.

**Evidence Expected:** Chatbot code, intent-level metrics, word-error-rate degradation table, threshold curve and an evidence-based deployment decision.

---

**MARK DISTRIBUTION – ASSIGNMENT 5**

| Part | Task | Assessed Area | Marks |
|---|---|---|---|
| A | A1 | Agent architecture design | 7 |
| A | A2 | Swarm design | 5 |
| A | A3 | Expert-system architecture | 5 |
| A | A4 | RDD pipeline design | 4 |
| A | A5 | Edge-model design | 4 |
| | | **Part A Total** | **25** |
| B | B1 | Drone agent implementation and measurement | 7 |

| Part | Task | Assessed Area | Marks |
|---|---|---|---|
| B | B2 | Expert-system engineering and adversarial testing | 5 |
| B | B3 | Applied case: full versus edge model | 8 |
| B | B4 | Conversational interface and deployment decision | 5 |
| | | **Part B Total** | **25** |
| | | **TOTAL** | **50** |

## SUBMISSION REQUIREMENTS

**Part A:** Problem formulation, agent/system definition, pseudocode, manual traces, design reasoning.

**Part B:** Source Code → Test Input → Expected Behaviour → Actual Output → Performance Metrics (latency, throughput, model size, accuracy, F1 and others as applicable) → Boundary/Failure Test → Interpretation.

Screenshots containing only a final output such as "Accuracy = 0.96" will not constitute sufficient evidence.

## SOFTWARE ENGINEERING EXPECTATIONS

Meaningful names, modular design, separation of agent logic from simulation and input/output, handling of boundary conditions, clear assumptions, fixed random seeds and reproducible test cases. A program that produces the expected output but whose logic cannot be explained or validated will not receive full credit.

## IMPORTANT RULE

**DO NOT STOP WHEN THE PROGRAM RUNS.** Establish: *Does the system work? → How many resources does it need and why? → When does it stop being practical? → What should the engineer do about it?*

**ENGINEERING WORKFLOW:** UNDERSTAND → DESIGN → ARCHITECT → IMPLEMENT → MEASURE → STRESS → COMPRESS → DECIDE → DEFEND

**Assessment Verification:** Students may be asked to give a short live demonstration and to explain any design, code, dataset split, metric or deployment decision in a brief viva.

# Solutions

**PART A – TASK A1: Design an Agent Architecture**

**A1.1 – PEAS Description**

- **Performance:** Safe delivery, minimum time and battery usage.
- **Environment:** City roads, buildings, obstacles and no-fly zones.
- **Actuators:** Motors, propellers and landing system.
- **Sensors:** GPS, camera, obstacle and battery sensors.

The environment is dynamic, partially observable and continuous. A hybrid agent using model-based, goal-based, utility-based and learning approaches is suitable.

**A1.2 – Percept's, State and Actions**

- **Percept's:** GPS location, obstacles, wind speed, battery level and destination.
- **Internal State:** Current position, planned route, battery status and obstacle locations.
- **Actions:** Move, turn, hover, avoid obstacles, deliver parcel and return to base.

**A1.3 – Decision Priorities**

Priority order:

1. Safety
2. Successful parcel delivery
3. Battery conservation

If an obstacle appears, the drone avoids it. If battery becomes low, it returns to base.

**A1.4 – Pseudocode**

```
BEGIN
  INITIALISE drone position, battery and destination
  WHILE mission is not complete
    SENSE environment
    UPDATE internal state
    IF obstacle or danger detected
      AVOID obstacle
    ELSE IF battery is below threshold
      RETURN TO BASE
    ELSE
```

PLAN route to destination

MOVE towards destination

END IF

UPDATE battery and position

IF parcel delivered OR emergency landing

TERMINATE mission

END IF

END WHILE

END

**A1.5 – Six-Step Trace**

| Step | Percept | Action |
|---|---|---|
| 1 | Clear path, battery 90% | Move forward |
| 2 | Wind detected | Adjust direction |
| 3 | Obstacle appears | Avoid obstacle |
| 4 | Path clear | Continue route |
| 5 | Battery reaches 20% | Return to base |
| 6 | Base detected | Land safely |

**Conclusion:** A hybrid drone agent combines planning and immediate reactions to ensure safe and successful delivery.

**PART A – TASK A2: Decentralised Swarm Design**

**A2.1 – Weakness of Centralised Design**

- Controller failure stops drone coordination.
- Communication failure reduces search coverage.
- High communication load causes delays.
- Mission success depends on one controller.

**A2.2 – Decentralised Swarm Strategy**

Each drone independently selects the nearest unexplored cell and avoids other drones.

**Pseudocode:**

BEGIN

SENSE nearby cells and drones

IF unexplored cell exists

SELECT nearest unexplored cell

MOVE towards selected cell

ELSE

MOVE to another available area

END IF

SHARE discovered locations with neighbours

END

**A2.3 – Example: 5 × 5 Grid**

Three drones start at different positions.

- Drone 1 searches the left area.
- Drone 2 searches the centre area.
- Drone 3 searches the right area.

They share explored locations and gradually cover the grid without central assignment.

**A2.4 – Failure Case**

If all three drones detect the same unexplored cell, they may move towards it together, causing clustering and repeated searches.

**Solution:** Use neighbour communication and assign different target cells.

**Conclusion:** Decentralised swarm systems improve coverage and continue working even if one drone or communication link fails.

**A3. Expert-System Architecture**

**A3.1 Rules for Three Expert Systems**

**1. Pump-Fault Diagnosis**

1. IF pressure is low, THEN check the pump.
2. IF motor temperature is high, THEN motor overheating.
3. IF vibration is high, THEN bearing fault.
4. IF pump does not start, THEN check the power supply.

**2. Crop-Disease Advisor**

1. IF leaves are yellow, THEN check for nutrient deficiency.
2. IF leaves have brown spots, THEN possible fungal disease.
3. IF leaves are wilting, THEN check water supply.
4. IF white powder appears on leaves, THEN possible powdery mildew.

**3. Flood-Relief Triage**

1. IF a person is injured, THEN assign high priority.
2. IF a person is trapped, THEN assign high priority.
3. IF food is unavailable, THEN assign medium priority.
4. IF people are safe with sufficient supplies, THEN assign low priority.

**A3.2 Common Expert-System Architecture**

All three systems use the same basic components:

- **Knowledge Base:** Stores rules and facts.
- **Working Memory:** Stores current input data.
- **Inference Engine:** Applies rules to reach conclusions.
- **Explanation Facility:** Explains the result.
- **Knowledge Acquisition:** Allows experts to update rules.

**Pseudocode:**

```
BEGIN
  GET user input
  STORE input in working memory
  MATCH input with knowledge-base rules
  APPLY matching rules
  RESOLVE conflicting rules
  DISPLAY conclusion
  EXPLAIN the result
END
```

**A3.3 Explanation and Knowledge Acquisition**

- **HOW explanation:** Shows how the system reached a conclusion.
- **WHY explanation:** Explains why a particular question or information is needed.
- **Knowledge acquisition:** Helps experts add or update rules, keeping the system accurate and useful.

**Conclusion:** A common expert-system shell can support different domains by changing only the knowledge base.

**A4. Distributed Data Pipeline Using RDDs**

**A4.1 RDD Operations**

- **Map:** Extract sensor ID and air-quality readings.
- **GroupByKey:** Group readings by sensor ID.

- **Reduce:** Calculate mean and maximum readings.
- **Sort:** Identify the ten noisiest sensors.

**A4.2 Transformations and Actions**

- Transformations: map(), filter(), groupByKey(), reduceByKey().
- Actions: collect(), count(), take().

**A4.3 Lazy Evaluation**

RDD transformations are not executed immediately. Spark executes them only when an action is called. This improves efficiency.

**A4.4 Shuffle and Caching**

- **Shuffle:** Transfers data between partitions during grouping and sorting.
- **Caching:** Stores frequently used RDDs in memory to avoid repeated calculations.

**A4.5 Fault Tolerance**

RDDs use lineage information to rebuild lost partitions if a failure occurs.

**Conclusion:** RDDs process large air-quality datasets efficiently through distributed computing and fault tolerance.

**A5. Edge-Model Design**

**A5.1 Selected Application:** Disaster Detection System.

A model detects disasters using images or sensor data and sends alerts quickly.

**A5.2 Why a Full Model Is Not Suitable**

- Requires high memory and processing power.
- Has higher execution time.
- May not meet the 50 ms latency limit.

**A5.3 Model Compression Techniques**

- **Pruning:** Removes unnecessary connections.
- **Quantisation:** Reduces numerical precision.
- **Knowledge Distillation:** Creates a smaller model from a larger model.
- **Feature Reduction:** Uses only important input features.

**A5.4 Simple Pseudocode**

```
BEGIN
  LOAD compressed detection model
  RECEIVE input data
  PROCESS input using edge model
  PREDICT disaster or normal condition
```

```
IF disaster detected
  SEND alert
ELSE
  CONTINUE monitoring
END IF
END
```

**A5.5 Trade-offs**

A smaller model reduces memory usage and response time but may slightly reduce accuracy.

**Conclusion:** An optimised edge model helps detect disasters quickly while aiming to meet the 50 ms latency requirement.

**B1. Drone Agent Implementation and Measurement**

**B1.1 Implementation**

Create a grid-based simulator where a drone moves from its starting point to a destination while avoiding obstacles and monitoring battery level.

**B1.2 Test Scenarios**

1. **Normal:** Drone reaches the destination safely.
2. **Obstacle:** Drone detects and avoids a sudden obstacle.
3. **Low Battery:** Drone returns to the base when battery is low.

**B1.3 Swarm Implementation**

Use multiple drones to search different grid cells. Each drone shares explored locations with nearby drones to reduce repeated searching.

**B1.4 Performance Measurement**

Test with 5, 10, 25 and 50 drones. Use different failure rates: 0%, 10% and 30%.

Measure:

- Mission success rate.
- Search coverage over time.
- Repeated coverage.
- Battery consumption.
- Response time.

**B1.5 Comparison**

Compare the single-drone and swarm approaches based on coverage, mission success and battery usage.

**Conclusion:** Simulation helps evaluate the efficiency, reliability and scalability of drone agents.

## B2. Expert-System Engineering and Adversarial Testing

### B2.1 Implementation

Develop a pump-fault diagnosis expert system using 15 rules covering five types of faults.

### B2.2 Testing

Prepare 30 expert-labelled test cases:

- 15 cases for training.
- 15 cases for testing.

Compare the system's predictions with expert results.

### B2.3 Performance Metrics

- Accuracy
- Precision
- Recall
- F1-score

### B2.4 Adversarial Testing

Test the system using:

- Conflicting rules.
- Missing input information.
- Incorrect or incomplete evidence.
- Different rule execution orders.

### B2.5 Conflict Resolution

Use rule priority and confidence levels to select the most suitable conclusion.

### B2.6 Rule Updates

Allow experts to add or modify rules. Run regression tests after every update.

**Conclusion:** Testing and rule updates improve the accuracy, reliability and flexibility of the expert system.

**B3. Full Model vs Edge Model**

**B3.1 Selected Application:** Disaster Detection System.

Use a suitable public dataset or simulator to train and test the detection model.

**B3.2 Full Model Evaluation**

Measure:

- Accuracy and F1-score.
- Model size and memory usage.
- Processing time.
- Five incorrect predictions.

**B3.3 Distributed Processing**

Use PySpark to process datasets containing 100,000, 1 million and 10 million records. Test with 1, 2, 4 and 8 partitions.

**B3.4 Edge Model Evaluation**

Apply pruning and quantisation to reduce model size and processing time. Measure average latency, 95th-percentile latency, throughput and accuracy.

**B3.5 Comparison**

Compare the full model and compressed edge model based on accuracy, memory usage and processing speed. Check whether the edge model meets the 50 ms latency limit.

**Conclusion:** The edge model is suitable for fast detection with limited resources, while the full model may provide better accuracy.

**B4. Conversational Interface and Deployment Decision**

**B4.1 Chatbot Development**

Develop a chatbot with at least eight intents using TF-IDF and cosine similarity to identify user queries.

**B4.2 Performance Evaluation**

Measure:

- Precision
- Recall
- F1-score
- Confusion matrix
- Response time

### B4.3 Speech Recognition Testing

Test the chatbot with different noise levels: 0%, 10%, 20% and 30%.

Measure intent accuracy, word error rate and incorrect responses.

### B4.4 Deployment Decision

Compare edge and cloud deployment.

- **Edge:** Faster response and better offline availability.
- **Cloud:** More computing power and easier model updates.

Choose deployment based on measured latency, accuracy and resource requirements.

**Conclusion:** A reliable chatbot should provide accurate responses, handle noisy inputs and use a suitable deployment method.